

Spike: Task 13**Title:** Composite & Component Patterns**Author:** Mohamed Shafi Uzman Fassy, 102608927**Goals / deliverables:**

The goal of this spike task is to extend our zorkish adventure game to create an immersive experience with the use of composite patterns and component patterns.

Besides this report, what else was created?

- Code files in see COS30031-2023-102608927\13 - Spike - Composite and Component Patterns\zorkish
- See modified .JSON file “world1” data structure.

Technologies, Tools, and Resources used:

List of information needed by someone trying to reproduce this work.

- Visual Studio 2022 Community Edition
- GitHub
- Task 12 Zorkish Code

Tasks undertaken:

- Set up task 12 zorkish project code into task 13 folder.
- Open it up Project Solution on Visual Studios.
- Modify .JSON file data structure to support entities and attributes.
- Modify Location to handle sublocations.
- Modify GameWorld to display available sublocations.
- Modify GoCommand to handle player movement into sublocations.
- Create Entities and Component Classes.
- Implement Component Attributes
- Create Extra Commands such as Open and Close Commands
- Compile Code and Test.

What we found out:

Composition Pattern

Since Composite Pattern involves creating objects in a tree structure to represent part-whole hierarchies, the objective was to enable clients to treat individual objects and composite objects uniformly. Therefore, I decided to implement sublocations within main locations by modifying our previous world1.json file from task 12.

```
{
  "locations": [
    {
      "name": "Mountain World",
      "desc": "peak of the highest mountain valley",
      "connections": [
      ],
      "inventory": [
      ],
      "sublocations": [
        {
          "name": "Rock Dungeon",
          "desc": "A dark, mysterious dungeon carved into the mountain.",
        }
      ]
    }
  ]
}
```

Figure 1: modified world1 Json file data structure.

I created a sublocation within Mountain World main location called Rock Dungeon in which players could move into only after they have moved into Mountain World giving it a name and description. This hierarchical relationship, where the main location contains sublocations, exemplifies the Composite Pattern.

```
if (data.contains("sublocations"))
{
    for (json subloc : data["sublocations"])
    {
        _sublocations.push_back(new Location(subloc));
    }
}
```

To do this, I modified the location class to check for the new sublocation we added to the game world so that when contrasting a location object

from the JSON file data structure and if does exist, it recursively creates a location object for each sub location. This helps because we might want to add multiple sublocations down the line.

```
else if (command.size() > 2 && command[1] == "into")
{
    // Handles moving into a sublocation
    string sublocationName = command[2];
    for (int i = 3; i < command.size(); i++) {
        sublocationName += " " + command[i];
    }

    Location* subLoc = pLoc->findSubLocation(sublocationName);
    if (subLoc)
    {
        player->setLocation(subLoc);
    }
    else
    {
        cout << "Invalid sublocation. Please choose another sublocation." << endl;
    }
}
```

Once that was done, we had to make sure the player could interact with sublocations and to this we had to extend our existing command from the previous task called **GoCommand** that basically acted as a command to move players between locations by saying player

types in “**go into <sublocation_name>**” that would let the player know they moved into an available sublocation.

```

Location* Player::getLocation()
{
    return _location;
}

string Player::getLocName()
{
    return _locName;
}

void Player::setLocName(const string& locName)
{
    _locName = locName;
}

void Player::setLocation(Location* loc)
{
    _location = loc;
}

void Player::enterSublocation(Location* subloc) {
    _currentSublocation = subloc;
}

void Player::exitSublocation() {
    _currentSublocation = nullptr;
}

Location* Player::getCurrentLocation() {
    return _currentSublocation ? _currentSublocation : _location;
}

```

However, for the player to know where they are located, I had to modify the player class to store their current location using the **getLocation()** method when they originally moved into the main location when they selected their adventure world and then updating the **getCurrentLocation()** method to then see if they are entering a sublocation or existing the sublocation. Therefore, by clearly demonstrating movement between a main location and a sublocation to facilitate interactions with the composite structure.

Finally, we check if the sublocation exists in the gameworld by checking the player's current location has any sublocations within it and if so, then display the available sublocations for the player to chose from and go into.

```

// Checks if the current location has any sublocations
vector<Location*> sublocations = currentLocation->getSubLocations();
if (!sublocations.empty())
{
    cout << "Available sublocations: " << endl;
    for (Location* sublocation : sublocations)
    {
        cout << "- " << sublocation->getName() << endl;
    }
}

```

Component Pattern

Since Component Pattern focuses on constructing objects from a system of behaviours instead of inheriting behaviour from a class hierarchy. Therefore, I modified the world1.json file which represents our game world once again and this time extended sublocations to manage entities with specific attributes and actions.

```

"sublocations": [
  {
    "name": "Rock Dungeon",
    "desc": "A dark, mysterious dungeon carved into the mountain.",
    "entities": [
      {
        "name": "Chest",
        "desc": "An old wooden chest. Wonder what's inside?",
        "attributes": {
          "state": "closed"
        },
        "actions": {
          "open": "The chest is now open.\n",
          "close": "The chest is now closed."
        }
      }
    ]
  }
]

```

For demonstration purposes I implemented a chest entity that has an attribute state which by default is closed and using actions it can change state to open / close the chest so that the game environment feels interactive.

Furthermore, to make change the attribute state using actions I created to Commands called Open Command and Close Command so that when the player enters the command “**open <entity name>**” for example the chest that exists in the sublocation Rock Dungeon it will change the attribute state to open and display on the console that the chest was open

```
if (args.size() > 1)
{
    Entity* entity = player->findEntityByName(args[1]);
    if (entity)
    {
        cout << entity->performAction("open");
    }
    else
    {
        cout << "Cannot find the entity to open.\n";
    }
}
else
{
    cout << "Specify what you want to open.\n";
}
```

```
Entity* entity = currentLocation->findEntityByName(entityName);
if (entity)
{
    string result = entity->performAction("close");
    cout << result << endl;
}
else
{
    cout << "Cannot find " << entityName << " to close.\n" << endl;
}
```

and if the player enters the command “**close <entity name>**” again for example the chest which is now in open state, will change back to close state and display that the chest was closed.

Finally, we again update the game world to find the chest entity to see if it exists within the current sublocation the player is currently located in and displays that a chest exists within the sublocation and for simplicity I hardcoded the find entity by name just to find an entity called chest.

```
// For entities in sublocation
Entity* entity = sublocation->findEntityByName("Chest");
if (entity)
{
    cout << " * " << entity->getName() << " - " << entity->getDescription() << endl;
}
```

Open issues/risks

Some of the issues I found out while testing out the game was that for some reason when I type in open chest or close chest it displays the chest attribute has changed which is intended but also displays the sublocation's description.

```
>> go into Rock Dungeon
A dark, mysterious dungeon carved into the mountain.
>> open chest
Cannot find the entity to open.
A dark, mysterious dungeon carved into the mountain.
>> close chest
Cannot find chest to close.
A dark, mysterious dungeon carved into the mountain.
>>
```

Also, another issue would be that I can not go back out of the sublocation to enter the main location and this could easily be fixed by implementing a go back command.